

Topic 5: Greedy Algorithms

Design and Analysis of Algorithms Laboratory

Exercise 5.1: Maximum Coins You Can Get (3n Piles)

Aim:

To maximize the total number of coins collected by picking the second largest pile in each optimal triplet from 3n piles.

Python Code:

```
def max_coins(piles):
    """
    Sort descending: Pick index 1, 3, 5, ... for n steps.
    Time Complexity: O(n log n), Space Complexity: O(1).
    """
    piles.sort()
    n = len(piles) // 3
    # We take every second pile from the top, skipping the smallest n piles
    return sum(piles[len(piles) - 2 - 2 * i] for i in range(n))

# Test Cases
print("piles = [2,4,1,2,7,8] ->", max_coins([2, 4, 1, 2, 7, 8]))
print("piles = [2,4,5] ->", max_coins([2, 4, 5]))
```

Sample Input:

Input: piles = [2, 4, 1, 2, 7, 8]

Input: piles = [2, 4, 5]

Sample Output:

Output: 9

Output: 4

Exercise 5.2: Minimum Coins to Make Range [1, Target] Obtainable

Aim:

To determine the minimum number of coins to add to an array so that every integer in [1, target] is formable by a subsequence sum.

Python Code:

```
def min_patches(coins, target):  
    """  
    Greedy reach expansion:  
    Time Complexity: O(len(coins) + log(target)), Space: O(1).  
    """  
    coins.sort()  
    miss = 1  
    added = 0  
    i = 0  
    n = len(coins)  
  
    while miss <= target:  
        if i < n and coins[i] <= miss:  
            miss += coins[i]  
            i += 1  
        else:  
            # Greedily add 'miss'  
            miss += miss  
            added += 1  
  
    return added  
  
# Test Cases  
print(min_patches([1, 4, 10], 19))  
print(min_patches([1, 4, 10, 5, 7, 19], 19))
```

Sample Input:

```
coins = [1, 4, 10], target = 19  
coins = [1, 4, 10, 5, 7, 19], target = 19
```

Sample Output:

```
Output: 2  
Output: 1
```

Exercise 5.3: Find Minimum Time to Finish All Jobs (k Workers)**Aim:**

To assign n jobs to k workers such that the maximum total working time across all workers is minimized.

Python Code:

```
def minimum_time_required(jobs, k):
    """
    Binary search on max working time + backtracking feasibility.
    """
    jobs.sort(reverse=True)
    low, high = max(jobs), sum(jobs)

    def can_assign(idx, cap, workers):
        if idx == len(jobs):
            return True
        j = jobs[idx]
        for w in range(k):
            if workers[w] + j <= cap:
                workers[w] += j
                if can_assign(idx + 1, cap, workers):
                    return True
                workers[w] -= j
            if workers[w] == 0:
                break
        return False

    ans = high
    while low <= high:
        mid = (low + high) // 2
        if can_assign(0, mid, [0] * k):
            ans = mid
            high = mid - 1
        else:
            low = mid + 1
    return ans

# Test Cases
print(minimum_time_required([3, 2, 3], 3))
print(minimum_time_required([1, 2, 4, 7, 8], 2))
```

Sample Input:

```
jobs = [3, 2, 3], k = 3
jobs = [1, 2, 4, 7, 8], k = 2
```

Sample Output:

```
Output: 3
Output: 11
```

Exercise 5.4: Maximum Profit in Job Scheduling

Aim:

To select a subset of non-overlapping jobs (by start and end times) that maximizes total profit using DP + Binary Search.

Python Code:

```
import bisect

def job_scheduling(startTime, endTime, profit):
    """
    Sort by end time and apply DP with binary search.
    Time Complexity: O(n log n), Space: O(n).
    """
    jobs = sorted(zip(startTime, endTime, profit), key=lambda x: x[1])
    dp = [(0, 0)] # (end_time, max_profit)

    for s, e, p in jobs:
        # Find latest job ending <= s
        idx = bisect.bisect_right(dp, (s, float('inf'))) - 1
        curr_profit = dp[idx][1] + p
        if curr_profit > dp[-1][1]:
            dp.append((e, curr_profit))

    return dp[-1][1]

# Test Cases
print(job_scheduling([1,2,3,3], [3,4,5,6], [50,10,40,70]))
print(job_scheduling([1,2,3,4,6], [3,5,10,6,9], [20,20,100,70,60]))
```

Sample Input:

```
startTime=[1,2,3,3], endTime=[3,4,5,6], profit=[50,10,40,70]
startTime=[1,2,3,4,6], endTime=[3,5,10,6,9], profit=[20,20,100,70,60]
```

Sample Output:

```
Output: 120
Output: 150
```

Exercise 5.5: Dijkstra's Algorithm (Adjacency Matrix)

Aim:

To compute the shortest paths from a source vertex to all vertices in a graph represented by an adjacency matrix.

Python Code:

```
def dijkstra_matrix(graph, src):
    """
    Dijkstra on adjacency matrix.
    Time Complexity:  $O(V^2)$ , Space Complexity:  $O(V)$ .
    """
    V = len(graph)
    dist = [float('inf')] * V
    visited = [False] * V
    dist[src] = 0

    for _ in range(V):
        # Pick min distance unvisited vertex
        u = -1
        for i in range(V):
            if not visited[i] and (u == -1 or dist[i] < dist[u]):
                u = i
        if dist[u] == float('inf'):
            break
        visited[u] = True
        for v in range(V):
            if graph[u][v] != float('inf') and not visited[v]:
                dist[v] = min(dist[v], dist[u] + graph[u][v])

    return dist

INF = float('inf')
g1 = [
    [0, 10, 3, INF, INF],
    [INF, 0, 1, 2, INF],
    [INF, 4, 0, 8, 2],
    [INF, INF, INF, 0, 7],
    [INF, INF, INF, 9, 0]
]
print("Test 1:", dijkstra_matrix(g1, 0))

g2 = [
    [0, 5, INF, 10],
    [INF, 0, 3, INF],
    [INF, INF, 0, 1],
    [INF, INF, INF, 0]
]
print("Test 2:", dijkstra_matrix(g2, 0))
```

Sample Input:

Test Case 1: n=5, source=0, graph with weights

Test Case 2: n=4, source=0, graph with weights

Sample Output:

Output: [0, 7, 3, 9, 5]

Output: [0, 5, 8, 9]

Exercise 5.6: Dijkstra's Algorithm (Edge List / Priority Queue)

Aim:

To find the shortest path from a given source vertex to a specific target vertex in a graph represented by an edge list.

Python Code:

```
import heapq
from collections import defaultdict

def dijkstra_edge_list(n, edges, source, target):
    """
    Time Complexity: O(E log V), Space: O(V + E).
    """
    adj = defaultdict(list)
    for u, v, w in edges:
        adj[u].append((v, w))
        adj[v].append((u, w))

    pq = [(0, source)]
    dist = {source: 0}

    while pq:
        d, u = heapq.heappop(pq)
        if u == target:
            return d
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if v not in dist or d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(pq, (d + w, v))

    return dist.get(target, -1)

# Test Cases
e1 = [(0, 1, 7), (0, 2, 9), (0, 5, 14), (1, 2, 10), (1, 3, 15), (2, 3, 11), (2, 5, 2), (3, 4, 6), (4, 5, 9)]
print("Test 1 (src=0, tgt=4):", dijkstra_edge_list(6, e1, 0, 4))

e2 = [(0, 1, 10), (0, 4, 3), (1, 2, 2), (1, 4, 4), (2, 3, 9), (3, 2, 7), (4, 1, 1), (4, 2, 8), (4, 3, 2)]
print("Test 2 (src=0, tgt=3):", dijkstra_edge_list(5, e2, 0, 3))
```

Sample Input:

Test 1: n=6, edges, source=0, target=4

Test 2: n=5, edges, source=0, target=3

Sample Output:

Output: 20

Output: 8

Exercise 5.7: Huffman Coding: Tree Construction and Code Generation

Aim:

To construct a Huffman Tree from characters and frequencies and generate prefix codes using a greedy priority queue approach.

Python Code:

```
import heapq

class HuffmanNode:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq

def build_huffman_codes(chars, freqs):
    pq = [HuffmanNode(c, f) for c, f in zip(chars, freqs)]
    heapq.heapify(pq)

    while len(pq) > 1:
        left = heapq.heappop(pq)
        right = heapq.heappop(pq)
        parent = HuffmanNode(None, left.freq + right.freq)
        parent.left = left
        parent.right = right
        heapq.heappush(pq, parent)

    root = pq[0]
    codes = {}
    def generate(node, current_code):
        if not node: return
        if node.char is not None:
            codes[node.char] = current_code
            return
        generate(node.left, current_code + "0")
        generate(node.right, current_code + "1")

    generate(root, "")
    return [(c, codes[c]) for c in chars]

# Test Cases
print("Test 1:", build_huffman_codes(['a','b','c','d'], [5, 9, 12, 13]))
print("Test 2:", build_huffman_codes(['f','e','d','c','b','a'], [5, 9, 12, 13, 16, 45]))
```

Sample Input:

```
n=4, characters=[a,b,c,d], frequencies=[5,9,12,13]
n=6, characters=[f,e,d,c,b,a], frequencies=[5,9,12,13,16,45]
```

Sample Output:

```
[('a', '110'), ('b', '10'), ('c', '0'), ('d', '111')]
[('a', '0'), ('b', '101'), ('c', '100'), ('d', '111'), ('e', '1101'), ('f', '1100')]
```

Exercise 5.8: Huffman Decoding Algorithm

Aim:

To decode a given Huffman encoded binary string back to its original message using the character code map / Huffman tree.

Python Code:

```
def huffman_decode(encoded_str, code_map):
    rev_map = {code: char for char, code in code_map}
    decoded = []
    curr = ""
    for bit in encoded_str:
        curr += bit
        if curr in rev_map:
            decoded.append(rev_map[curr])
            curr = ""
    return "".join(decoded)

# Test Cases
codes1 = [('a', '110'), ('b', '10'), ('c', '0'), ('d', '111')]
print("Decoded 1:", huffman_decode('1101100111110', codes1))

codes2 = [('a', '0'), ('b', '101'), ('c', '100'), ('d', '111'), ('e', '1101'), ('f', '1100')]
print("Decoded 2:", huffman_decode('110011011100101111001011', codes2))
```

Sample Input:

Test 1: encoded_string = '1101100111110'
 Test 2: encoded_string = '110011011100101111001011'

Sample Output:

Output: "abacd"
 Output: "fcbade"

Exercise 5.9: Container Loading: Maximize Weight (Heavier First)

Aim:

To load the maximum total weight into a container using a greedy strategy that prioritizes heavier items first.

Python Code:

```
def container_loading_max_weight(weights, capacity):
    """
    Greedy heuristic: Sort descending and load items that fit.
    """
    sorted_w = sorted(weights, reverse=True)
    total_loaded = 0
    for w in sorted_w:
        if total_loaded + w <= capacity:
            total_loaded += w
    return total_loaded

# Test Cases
print("Test 1:", container_loading_max_weight([10, 20, 30, 40, 50], 60))
print("Test 2:", container_loading_max_weight([5, 10, 15, 20, 25, 30], 50))
```

Sample Input:

Test 1: weights = [10, 20, 30, 40, 50], max_capacity = 60
 Test 2: weights = [5, 10, 15, 20, 25, 30], max_capacity = 50

Sample Output:

Output: 50 (Loads 50)

Output: 50 (Loads 30 + 20 = 50)

Exercise 5.10: Container Loading: Minimize Number of Containers**Aim:**

To pack all items into the minimum number of equal-capacity containers using greedy Next-Fit / First-Fit heuristic.

Python Code:

```
def min_containers_required(weights, capacity):
    """
    First-Fit Decreasing heuristic.
    """
    sorted_w = sorted(weights, reverse=True)
    bins = []
    for w in sorted_w:
        placed = False
        for i in range(len(bins)):
            if bins[i] + w <= capacity:
                bins[i] += w
                placed = True
                break
        if not placed:
            bins.append(w)
    return len(bins)

# Test Cases
print("Test 1:", min_containers_required([5, 10, 15, 20, 25, 30, 35], 50))
print("Test 2:", min_containers_required([10, 20, 30, 40, 50, 60, 70, 80], 100))
```

Sample Input:

Test 1: weights = [5, 10, 15, 20, 25, 30, 35], max_capacity = 50

Test 2: weights = [10, 20, 30, 40, 50, 60, 70, 80], max_capacity = 100

Sample Output:

Output: 4

Output: 6

Exercise 5.11: Kruskal's Minimum Spanning Tree Algorithm

Aim:

To find the Minimum Spanning Tree (MST) and its total weight in an undirected graph using Kruskal's algorithm and Disjoint Set Union (DSU).

Python Code:

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, i):
        if self.parent[i] == i:
            return i
        self.parent[i] = self.find(self.parent[i])
        return self.parent[i]

    def union(self, i, j):
        root_i = self.find(i)
        root_j = self.find(j)
        if root_i != root_j:
            self.parent[root_i] = root_j
            return True
        return False

def kruskal_mst(n, edges):
    edges.sort(key=lambda x: x[2])
    dsu = DSU(n)
    mst = []
    total_weight = 0

    for u, v, w in edges:
        if dsu.union(u, v):
            mst.append((u, v, w))
            total_weight += w

    return mst, total_weight

# Test Cases
e1 = [(0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4)]
mst1, w1 = kruskal_mst(4, e1)
print(f"MST 1: {mst1}, Total Weight: {w1}")

e2 = [(0, 1, 2), (0, 3, 6), (1, 2, 3), (1, 3, 8), (1, 4, 5), (2, 4, 7), (3, 4, 9)]
mst2, w2 = kruskal_mst(5, e2)
print(f"MST 2: {mst2}, Total Weight: {w2}")
```

Sample Input:

Test 1: n=4, edges = [(0,1,10), (0,2,6), (0,3,5), (1,3,15), (2,3,4)]
 Test 2: n=5, edges = [(0,1,2), (0,3,6), (1,2,3), (1,3,8), (1,4,5), (2,4,7), (3,4,9)]

Sample Output:

Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)], Total weight: 19
 Edges in MST: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)], Total weight: 16

Exercise 5.12: MST Uniqueness Verification

Aim:

To determine whether a graph's Minimum Spanning Tree is unique, and if not unique, construct an alternative valid MST.

Python Code:

```
def check_mst_uniqueness(n, edges, given_mst):
    # An MST is unique iff no edge not in MST can replace an edge in MST with equal weight
    mst_weight = sum(w for _, _, w in given_mst)
    mst_set = set((min(u, v), max(u, v), w) for u, v, w in given_mst)

    alternative = None
    for u, v, w in edges:
        norm = (min(u, v), max(u, v), w)
        if norm not in mst_set:
            # Check if including this edge produces another valid MST of equal weight
            dsu = DSU(n)
            dsu.union(u, v)
            cand_mst = [norm]
            cand_weight = w
            for u2, v2, w2 in sorted(edges, key=lambda x: x[2]):
                if dsu.union(u2, v2):
                    cand_mst.append((min(u2, v2), max(u2, v2), w2))
                    cand_weight += w2
            if len(cand_mst) == n - 1 and cand_weight == mst_weight:
                alternative = cand_mst
                break

    is_unique = alternative is None
    return is_unique, alternative

# Test Cases
e1 = [(0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4)]
m1 = [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
print("Test 1 Unique?", check_mst_uniqueness(4, e1, m1)[0])

e2 = [(0, 1, 1), (0, 2, 1), (1, 3, 2), (2, 3, 2), (3, 4, 3), (4, 2, 3)]
m2 = [(0, 1, 1), (0, 2, 1), (1, 3, 2), (3, 4, 3)]
uniq, alt = check_mst_uniqueness(5, e2, m2)
print(f"Test 2 Unique? {uniq}, Alternative: {alt}")
```

Sample Input:

Test 1: Given MST = [(2,3,4), (0,3,5), (0,1,10)]

Test 2: Given MST = [(0,1,1), (0,2,1), (1,3,2), (3,4,3)]

Sample Output:

Is the given MST unique? True

Is the given MST unique? False, Another possible MST: [(0, 1, 1), (0, 2, 1), (2, 3, 2), (3, 4, 3)], Total weight: 7